

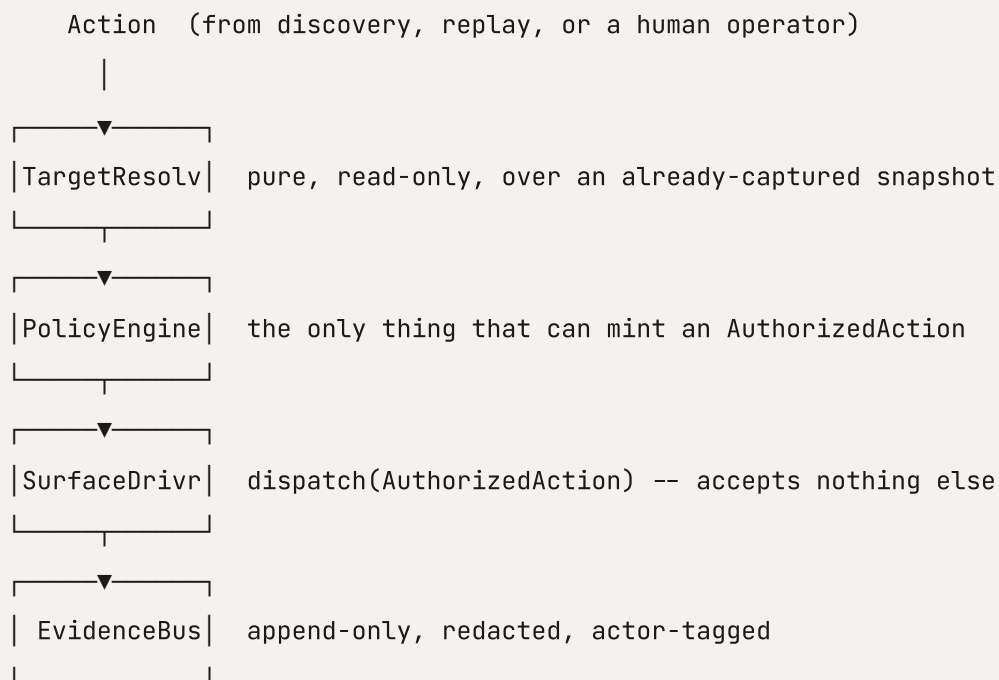
Design write-up

Discovery is probabilistic. Execution is deterministic. The model is allowed to be uncertain exactly once. That uncertainty is compiled into an artifact that never needs it again.

Every claim below is backed by something runnable: an enforced import contract, a test, or a file in [evidence/](#). What is not built says so in §7.

1. Architecture

An LLM drives a live legacy UI to a goal **once**. That run is compiled into a typed, versioned **capability artifact**. From then on the artifact is replayed with no model in the decision loop, and an AI agent invokes it like a typed function.



Discovery, replay, and human intervention are the same path, differing only in who originates the action. That is the answer to "how do you know your guardrails apply in production?" — there is one path, so a guardrail cannot be present on one and missing from another.

Resolution runs **before** policy. I documented it the other way round and implementing it proved me wrong: risk classification's second signal is *what the control says it does* ("Transfer Funds"), which needs the resolved node, so authorizing first means authorizing half-blind. The invariant was never "policy runs first" — it is *nothing reaches a surface without authorization*.

Three enforcements, because an invariant nobody can violate beats one everybody agrees with: an **import contract**, a **type guard** (`dispatch()` takes only an `AuthorizedAction`), and an **AST scan** that has caught four violations the import graph missed. Every fix moved the module; none added an exemption.

Stack. Python for Pydantic v2, which lets the artifact schema *be* the type system rather than sit beside it. Playwright over CDP because the accessibility tree is the closest browser analogue to what UIA or AX exposes, so perception ports. The model sits behind an `LlmPort` protocol, so the provider is one class rather than a dependency; `temperature=0`. YAML artifacts because a bank reviewer has to read one without running anything.

2. Artifact schema

The focal point. `cua.capability/v1` — Pydantic, YAML, immutable, content-addressed. Examples in [evidence/capabilities/](#).

Targets are semantic, never selectors. A `TargetDescriptor` is role, accessible name, structural anchor, scope, ordinal. A CSS selector may appear only under `hints` — a cache accepted only if the node it finds *also* satisfies the semantic assertion. `role: button, name: "Search"` means the same in Windows UIA as in ARIA; `input.btn1` does not.

Business outcomes, recovery, and failure are three distinct top-level keys — the brief names conflating the first with the third as the most common mistake here, so the schema makes it structurally impossible rather than trusting the executor.

The capability is immutable; everything a *run* produces lives elsewhere — `RunRecord`, `CapabilityApproval`, `TenantBinding`, keyed by `id@version`. A definition that accumulates telemetry stops being reviewable, and "which version ran?" stops being answerable.

`capability.id` is namespaced by vendor product, not tenant, and `surface.driver_capabilities` declares what a driver must *be able to do* rather than naming a library. Both are preconditions for reuse.

The compiler emits a **draft** and says so in the artifact: one run saw only the happy path, so it declares no outcomes and no recovery, with review notes naming what a reviewer must add — inventing detectors a run never observed produces error handling that is fiction.

3. Determinism & error handling

Two type families, deliberately separate — *what we observed* and *how the run ended*:

OBSERVATIONCLASS	MEANING	LEADS TO
EXPECTED	matches a precondition/checkpoint	proceed
BUSINESS_OUTCOME	matches a declared outcome	terminate, exit 0
RECOVERABLE	matches a declared recovery rule	bounded remedy, then retry
UNEXPECTED_STATE	matches nothing declared	fail closed

`RunStatus` is terminal and separate: `SUCCESS` | `BUSINESS_OUTCOME` | `NEEDS_HUMAN` | `FAILED` .

"No such member" **exits 0**. It is a successful execution returning a negative answer; treating it as a failure turns a routine result into a 2am page and trains everyone to ignore the alert that also fires for real defects.

Fail-closed is the default. A state matching nothing declared stops the run; it never proceeds assuming the click probably worked. Two surviving candidates is `TARGET_AMBIGUOUS` and the resolver **refuses** — there is no similarity threshold to tune, because a tunable threshold is a knob that eventually gets turned.

Determinism is enforced, not asserted: `cua.replay` may not import `cua.agent` or any HTTP client; the ladder is a fixed order with pure scoring, ties broken by document order, never by dict iteration, clock or random; the `vision` rung is hard-disabled in replay.

Where a hard failure goes is the artifact's decision, not the executor's —

`escalation.triggers` decides whether it pauses for a human or terminates. Writing the demo I expected an exhausted recovery to come back `FAILED` and it came back `NEEDS_HUMAN` ; the artifact was right and I was wrong, which is the point of putting the routing there.

Drift is measured as descent. A target recorded at `semantic_exact` that now resolves only at `structural_anchor` has moved, even though the run still succeeded — so `run_record.drift_score` rises before replays begin to fail, rather than after. `make eval` reports determinism as a boolean per capability, because "94% deterministic" is not a property anyone can act on.

4. Heterogeneity & multi-tenant

The seam is `UiSnapshot` — a normalized `{node_id, role, name, value, states, bounds, scope}` tree. Drivers populate it from whatever their surface offers: a CDP accessibility tree today, Windows UIA tomorrow. `cua.targeting` and `cua.perception` **may not import Playwright or `cua.surfaces`** (enforced), so neither the artifact nor the resolver can grow web-shaped assumptions. `surfaces/desktop_uia/` is an interface-only stub documenting the `UIA`→`UiSnapshot` mapping.

Tenants are an **overlay, never a fork**: `TenantBinding` supplies vars and per-step overrides, resolved at load into an effective capability whose content hash covers the overlay.

The no-CSS claim is **measured**, in [evidence/evals/cross_tenant.md](#). Variant B relabels controls *and* restyles markup, so every `hints.css` in the artifact names a selector that does not exist there — and the runs succeed anyway: 100%, zero wrong actions. Markup churn is carried by `semantic_exact`; rebranding by `structural_anchor` plus the overlay. Both rungs are load-bearing, which is why the ladder has both.

A Phase-01 spike **disproved my own claim**: `structural_anchor` does not survive rebranding, because the anchor text *is* the label cell and gets relabeled too — so the honest claim is markup change, not vocabulary change, and vocabulary is what the overlay is for. One limit that leaves: drift is computed against the *effective* capability, so an overlay reports zero drift for a surface that genuinely differs.

5. Escalation & handoff

`NEEDS_HUMAN` is a **first-class terminal status**, not an exception — which is what makes escalation architectural rather than bolted on.

The operator works the **same live session**, guarded by a `Lease{holder, epoch}`. Automation consults it twice per dispatch: **who holds this session**, and **has it changed hands since this run was authorized**. Either returns `LEASE_LOST`. Both, because the epoch alone is not enough — and assuming it was is how this stayed broken: the check is opt-in, the executor never asked for it, and two tests calling the dispatcher directly made it look covered. Before the fix a replay ran to `SUCCESS`, reading a balance, while an operator held the session.

Human input does not bypass policy. The console never injects events into the page. It submits `raw_input` actions to the broker, which runs them through the same resolve → authorize → dispatch → evidence path under a **HUMAN profile**. Escalation widens authority deliberately and auditably; the allowlist and redaction apply unchanged.

Control comes back. `ReplayExecutor.resume()` re-anchors against the live screen and continues. It is not `run()` with an offset: it skips the entrypoint navigation (which would reload the frameset and discard the state the operator just produced), re-adopts the epoch the handoff produced (a full cycle advances the lease four times, so the run's own epoch is stale by construction), carries forward outputs and spent recovery attempts, and fails closed when the screen matches no step. A step is skipped only when its **postcondition** holds — which is why `submit_search` has one; without it a resume re-submits the search the operator just ran.

`cua console --capability <ref> --arm-fault <fault>` runs a capability on the supervised session, so the queue a reviewer opens has something in it.

6. Safety

One chokepoint, three enforcements (§1), applied identically to all three actors.

Allowlist — domains, URL patterns, action types, step and duration caps, in `config/policy.yaml`. Navigation outside it is `NAVIGATION_BLOCKED`, for humans too. A capability's own `allowed_domains` **narrows** the global list. Two gaps closed here, both the same shape — a check that covered the stated intent and missed the actual outcome. Entrypoint navigation now goes through the ordinary path rather than the raw page handle,

because `--target` is caller-supplied. And the allowlist is re-checked against where the session *landed* after any action that navigated, not only against a `navigate`'s declared URL: a click on a link, on a page whose content is untrusted, carries no URL to check up front.

Risk tiers — `safe` → `elevated` → `irreversible`, from three independent signals (action type × target semantics × artifact annotation), because any one alone is fooled. Automation is blocked from irreversible actions, and replay requires an approved capability **and** a caller opt-in — two gates, because either alone is one accident away from a wire transfer.

Redaction at every sink, including outbound model prompts. "Every sink" is load-bearing and has been untrue twice. `run_record.json` was once written straight to disk under different rules than the trace beside it; `write_run_record` now *requires* a redactor. And `Redactor.register_secret` — the literal path that exists because no pattern recognises an arbitrary password — had no production caller: only tests called it, so `extra_secrets` was empty in every real run. `SecretResolver.resolve()` now registers each value as it issues it, and the invariant test asserts through the resolver instead of performing the registration itself.

Over-redaction is a failure too: `id@version` matched the email pattern, so every run record read `<redacted:email>` in the one field naming which capability ran. Tested both directions now.

The operator's live view is redacted the same way an evidence screenshot is, from the supervised capability's own `sensitive` declarations — it was the one sink showing regulated data in the clear, and the one most likely to be on a second monitor. A hold that lapses is reclaimed to `PAUSED` and its intervention returned to the queue as `ABANDONED`; the expiry the lease describes previously had no caller, so a session an operator walked away from stayed pinned forever.

Limits, stated rather than claimed away. Free-text redaction is patterns plus registered literals, so a member *name* is protected only where a capability declares the field `sensitive`, not by shape. Screenshot redaction has the same dependency: it can only paint over what a declaration can locate. Prompt injection is scoped, not solved — closed action space, policy evaluated *outside* the model, no authority to widen the allowlist — but a model talked into a *permitted* action on a *permitted* target still performs it. Fixture sign-in (`--sign-in`, the demo, the eval harness) drives the browser directly: the one documented carve-out, opt-in and scoped to the mock app's fixed credentials. No real credentials or PII exist in this repository.

7. Cuts

Built after the critical path. `cua eval` turned §4's central claim into a measurement and immediately falsified part of it: an overlay could retarget a control but not restate the step's precondition or the checkpoint, so every Variant B run failed closed on an assertion written for a different bank. `StepOverride` now carries them. The catalog refuses a tampered artifact while still *listing* it as `TAMPERED`, because an operator whose artifact was edited needs telling.

Scope the brief did not ask for. §3.6 scopes a co-browsing console out and invites a mock; I built a working one because control transfer is the part I most wanted to prove end to end. That is ~5,000 lines nobody asked for, plus a docs site and a UI/UX guide that are presentation, not system design. Cutting to the brief's stated preference for "a small, correct, well-argued system", those go first.

Still cut, in restore order. Continuous pixel streaming — the console sends a still frame on connect and after each policed gesture, enough to see and act on the page but not co-browsing. A desktop surface — interface-only stub on purpose; declaring the seam and enforcing the import rule is what it is worth here. Multi-operator routing, SSO, audit sign-off — designed, not built. Assisted LLM fallback on replay failure — would breach the no-model-in-replay invariant without a separate policed path.

The discovery evidence is a real model run. <evidence/discovery/disc-0a7e7b7ccd/> is a genuine LLM-driven run against the live frameset app, routed through a self-hosted OpenAI-compatible gateway: six model calls, 10,215 tokens, each carrying the gateway's own provider and request id so the run can be checked against the gateway's logs rather than taken on trust. The model signed in, searched, read the balance, status and as-of date off the member record and stopped -- three effective steps. The artifact the compiler produced from it, `memberdesk.savings_balance@1.0.0`, replays for members that run never saw: 12345, 67890 and the closed account 24680, model-free, at zero drift.

Publication is the part worth stating precisely, because getting it wrong would quietly turn that claim into a fiction. `cua discover` publishes into the catalog; `make demo` never publishes over an artifact that is already there. The condition is the catalog's, not the model's -- an earlier version gated on whether a model had run, which held only until a key was configured and the demo went live too, at which point it republished over the discovery it was supposed to protect. A fresh clone with an empty catalog still sees record → compile → publish → replay end to end.

Known weaknesses. The compiler describes a value by the label beside it — right on a label/value table, wrong in an n-column grid where the neighbour is another datum. It now refuses a datum-shaped anchor and falls to a positional descriptor flagged for review, but cannot yet read a *column header*, which is the correct answer there. `raw_input` is authorized as a *declared tier* (`elevated` in `config/policy.yaml`) rather than by classifying what it touches: a mouse-move has no resolved target to reason about. That is defensible for drag and scroll, but it means a human's raw click is authorized far more coarsely than a semantic action, and the allowlist is checked on a `navigate` 's *intent* rather than on where the session actually lands — a click that navigates is not re-checked afterwards. `config/policy.yaml` 's `budgets` and `replay_gates` are parsed and displayed but not yet read by the executor.

What to look at

The artifact	<code>evidence/capabilities/</code>
The handoff, with actors and epochs	<code>evidence/escalation/demo-escalation/trace.jsonl</code>
All four terminal statuses	<code>evidence/replay/</code> , <code>evidence/escalation/</code>
The invariants, enforced	<code>.importlinter</code> , <code>tests/invariants/</code>
The whole story, one command	<code>make demo</code>
The numbers behind §3 and §4	<code>evidence/evals/</code>
